Universidad Nacional Autónoma de México
Facultad de Ingeniería

Práctica 2.

Silverio Martínez Andrés

Lab. Organización y Arquitectura de Computadoras

Grupo: 8

Semestre 2026-2

Profesor: Jorge Ferat Toscano

Fecha de entrega: 14 de marzo de 2026

Práctica 2

Objetivo

Implementar y simular en Quartus Prime un sumador completo de un bit utilizando el lenguaje VHDL, verificando su funcionamiento mediante la tabla de verdad y las herramientas de simulación, con el fin de comprender el proceso de diseño digital desde la descripción del hardware hasta la validación del circuito lógico.

Introducción

El diseño digital es una de las bases fundamentales en el campo de la electrónica y los sistemas embebidos, ya que permite implementar circuitos lógicos que sirven como bloques constructivos de sistemas más complejos. En esta práctica se estudió el funcionamiento de un **sumador completo de un bit**, circuito combinacional que suma tres bits de entrada (a0, b0 y c0) y genera como resultado una suma (s0) y un acarreo (c1).

Para el desarrollo se utilizó **Quartus Prime**, software de diseño asistido por computadora (CAD) que permite describir hardware en lenguajes como VHDL y verificar su comportamiento mediante herramientas como el **RTL Viewer** y el **Simulation Waveform Editor**. A través de la simulación con señales configuradas como relojes, se recorrieron todas las combinaciones posibles de entradas, comprobando que las salidas cumplen con la tabla de verdad teórica del sumador completo.

Esta práctica no solo refuerza el entendimiento de los circuitos combinacionales, sino que también introduce al estudiante al flujo de trabajo típico en diseño digital: creación del proyecto, codificación en VHDL, compilación, análisis en RTL Viewer y simulación de resultados.

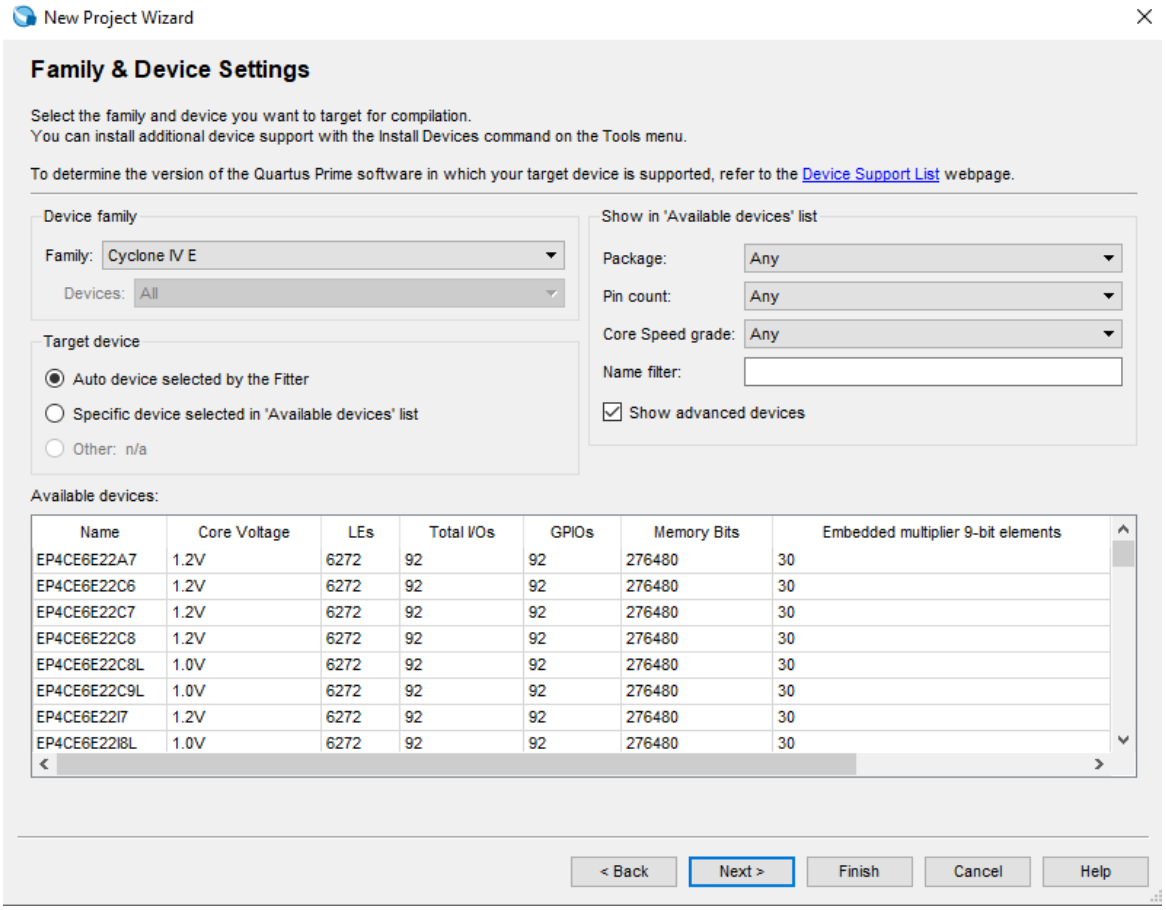
Desarrollo

1) Crear un proyecto con nombre practica 2, utilizando la misma forma de generarlo conforme a lo realizado en la anterior practica 1
Paso 1: Seleccionamos la opción de “New Project Wizard”

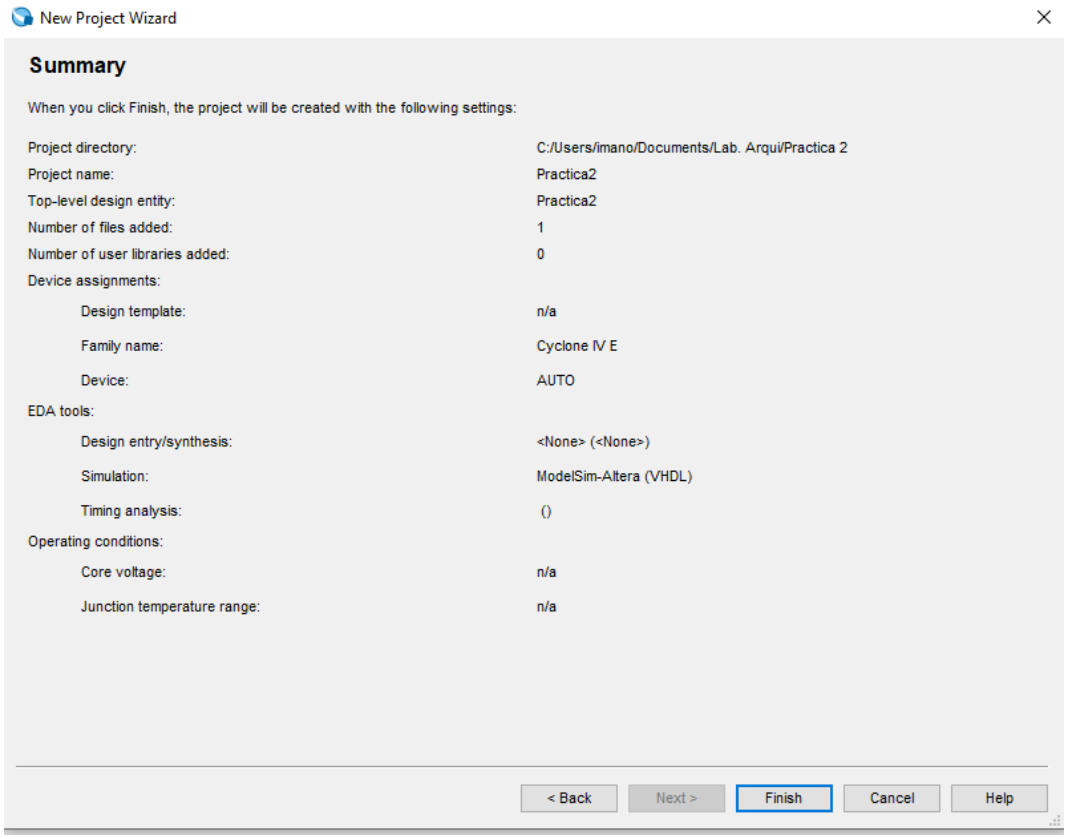
Paso 2: Seleccionamos la carpeta donde queremos guardar el proyecto

Paso 3: Le asignamos el nombre del proyecto

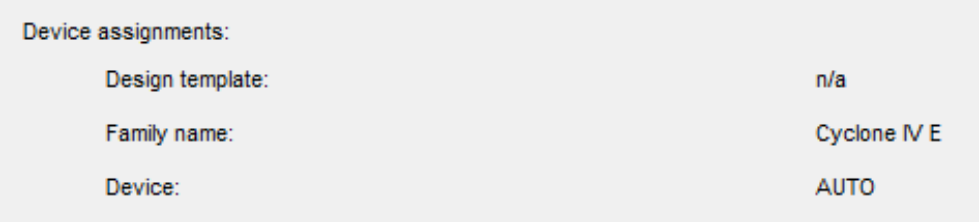
Paso 4: Llegamos a esta ventana y ponemos en auto



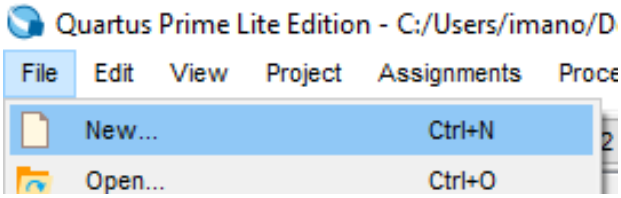
Paso 5: Para terminar, nos aparecerá la siguiente ventana y le daremos clic en “Finish”



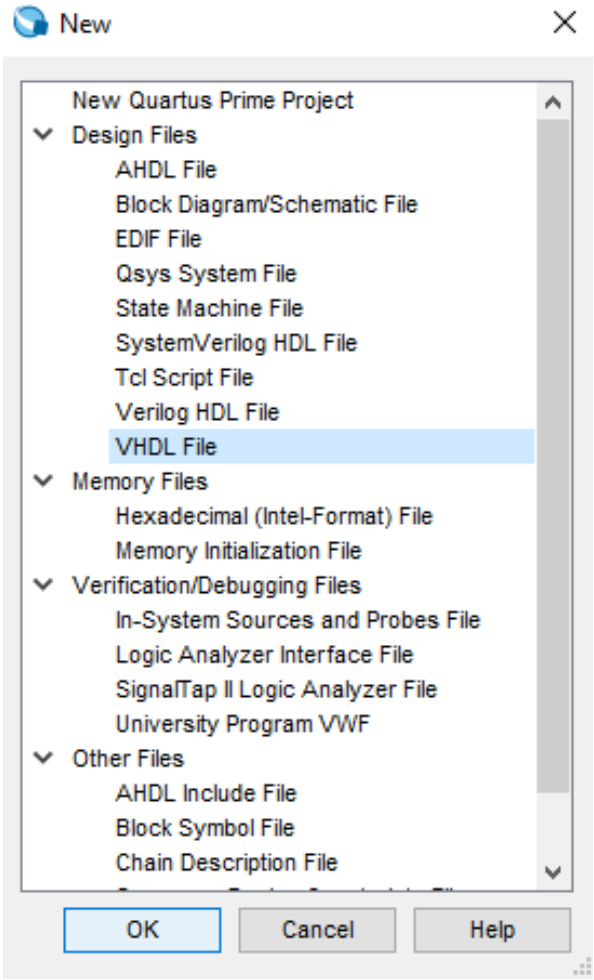
2) El tipo de dispositivo a utilizar considere Auto
En la siguiente imagen vemos, que pusimos Auto en la configuración:



3) Crear un archivo nuevo de diseño y se va escoger el medio VHDL, (sumador)
Paso 1: Damos clic en “File” y después en “New”



Paso 2: En la siguiente ventana, dar clic en “VHDL File” y después clic en “Ok”



4) Una vez que este generado teclear en el editor el código

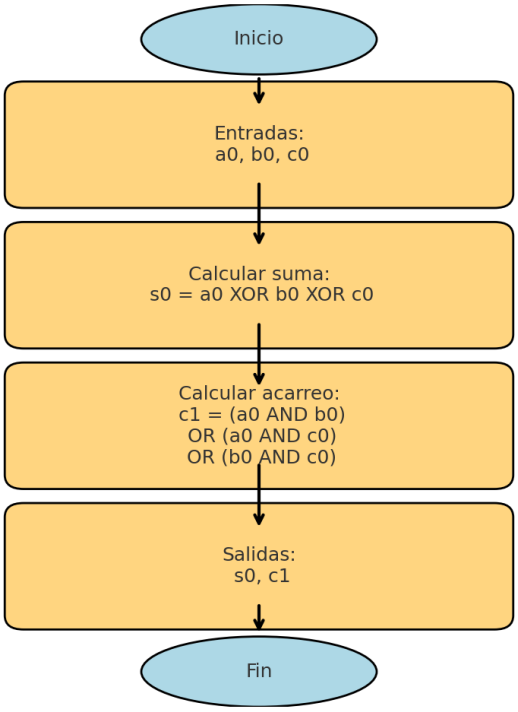
```vhdl
-- Declaración de la entidad llamada "practica2"
-- Define las entradas y salidas del circuito
entity practica2 is
 port (
   a0, b0, c0 : in bit;  -- Entradas: dos operandos (a0, b0) y un acarreo de entrada (c0)
   c1, s0     : out bit  -- Salidas: suma (s0) y acarreo de salida (c1)
 );
end practica2;

-- Descripción de la arquitectura "suma" para la entidad "practica2"
architecture suma of practica2 is
begin
  -- Asignación concurrente para la salida de la suma
  -- s0 representa el bit de la suma resultante
  -- La operación XOR permite calcular la suma de los bits de entrada con el acarreo
  s0 <= a0 xor b0 xor c0;

  -- Asignación concurrente para la salida del acarreo
  -- c1 se activa cuando al menos dos de las entradas son '1'
  -- Esta es la expresión booleana clásica del acarreo en un sumador completo
  c1 <= (a0 and b0) or (a0 and c0) or (b0 and c0);
end suma;
```

Explicación:

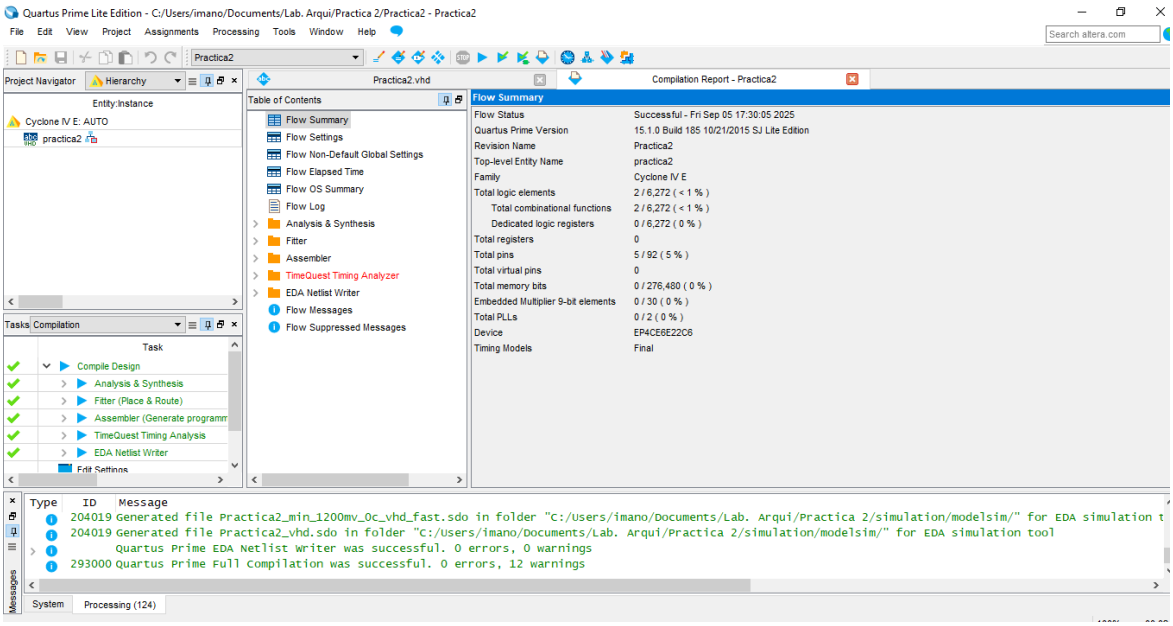
- La **entidad (`entity`)** define los pines del módulo:
  - `a0` y `b0`: los bits de entrada que se quieren sumar.
  - `c0`: el bit de acarreo de entrada.
  - `s0`: la salida que corresponde a la suma.
  - `c1`: el acarreo de salida que se genera.
- La **arquitectura (`architecture`)** describe el comportamiento del circuito:
  - `s0` se calcula con una operación XOR de las tres entradas, lo cual es la regla del sumador completo para la suma.
  - `c1` se obtiene de las combinaciones donde dos entradas o más son 1, lo que genera el acarreo.

En otras palabras, este código **implementa un sumador completo de 1 bit en VHDL**, que es un bloque fundamental para construir sumadores de mayor tamaño en FPGAs o circuitos digitales.

Diagrama de flujo:

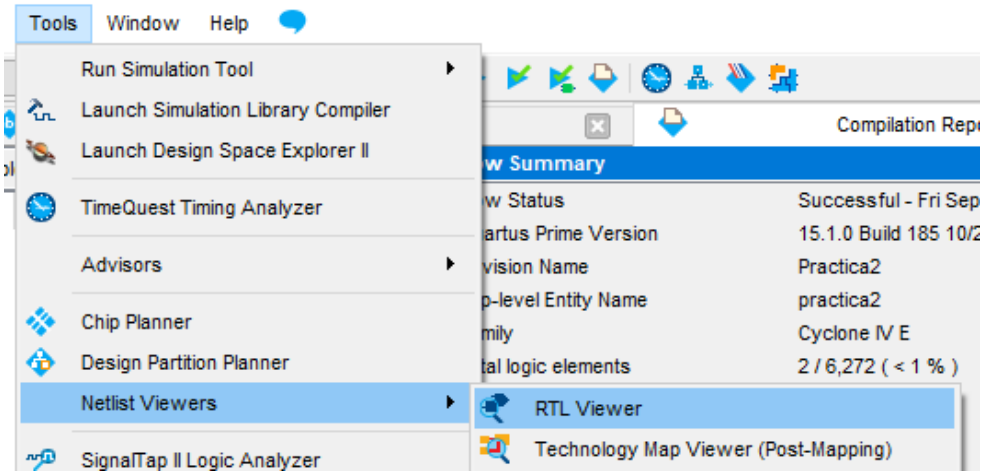


5) Compilar el programa Revisar que la compilación fue de manera **exitosa**

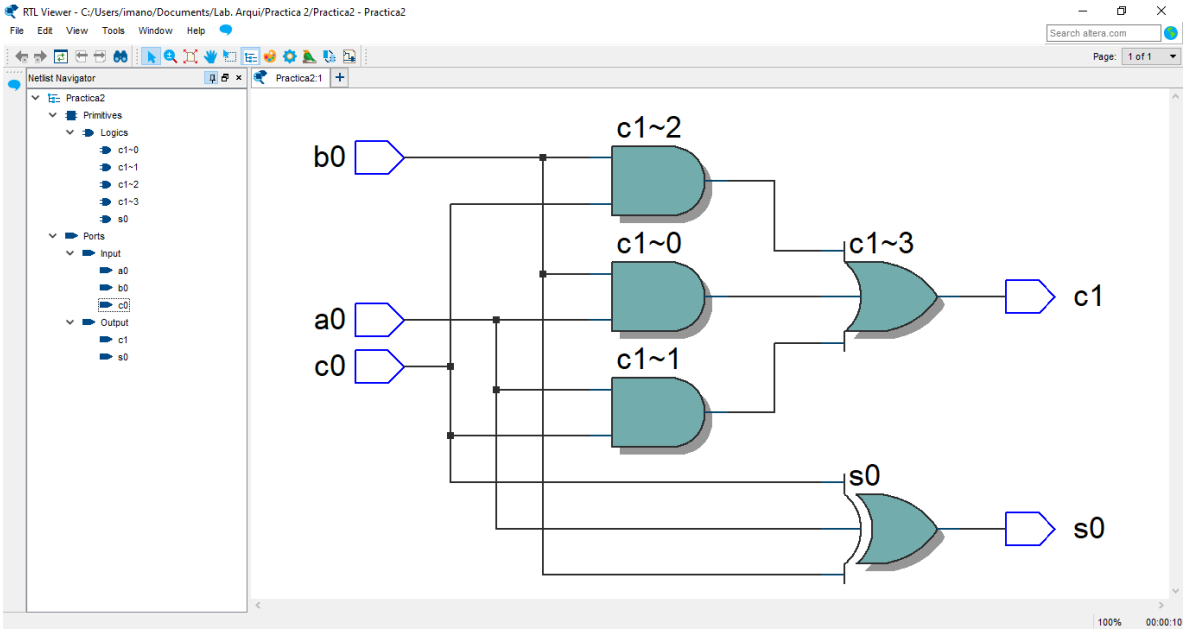


6) Procedemos a revisar el circuito sintetizado (RTL Viewer) Se observará en la pantalla el diagrama correspondiente al circuito

Paso 1: Nos dirigimos en la pestaña de “Tools”, buscamos “Netlist Viewers” y finalmente damos clic en “RTL Viewer”



Este es el circuito generado:

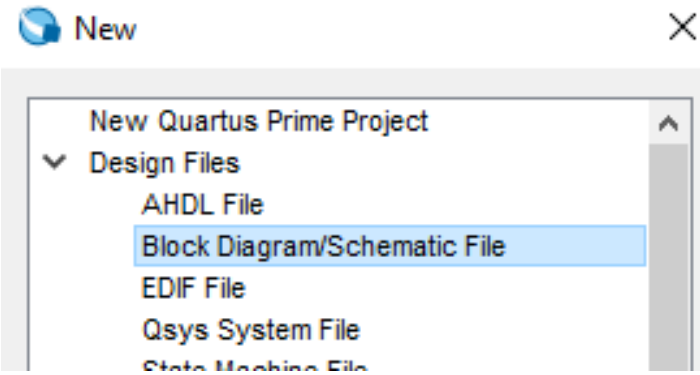


El **RTL Viewer** en Quartus te muestra el **esquema lógico sintetizado** (el circuito que el compilador generó a partir de tu código VHDL). Pero **el RTL Viewer no es un simulador**, solo es una herramienta de verificación visual del diseño.

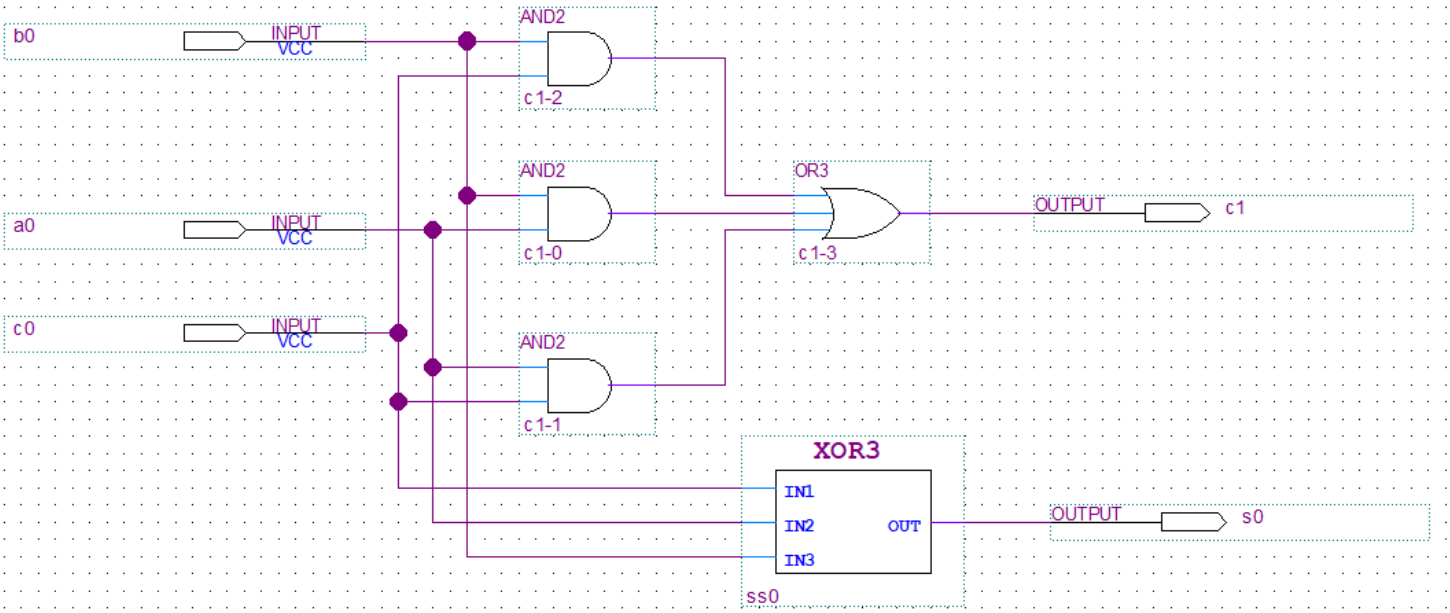
Entonces hay que simularlo y ver si es correcto.

7) Una compilación exitosa no garantiza un funcionamiento satisfactorio, pues podrían existir errores de lógica. Por lo anterior es conveniente simular comportamiento del circuito. Para ello se procederá a realizar la simulación del mismo

Paso 1: Seleccionamos “File” luego “New” y damos clic en “Block Diagram/Schematic File”



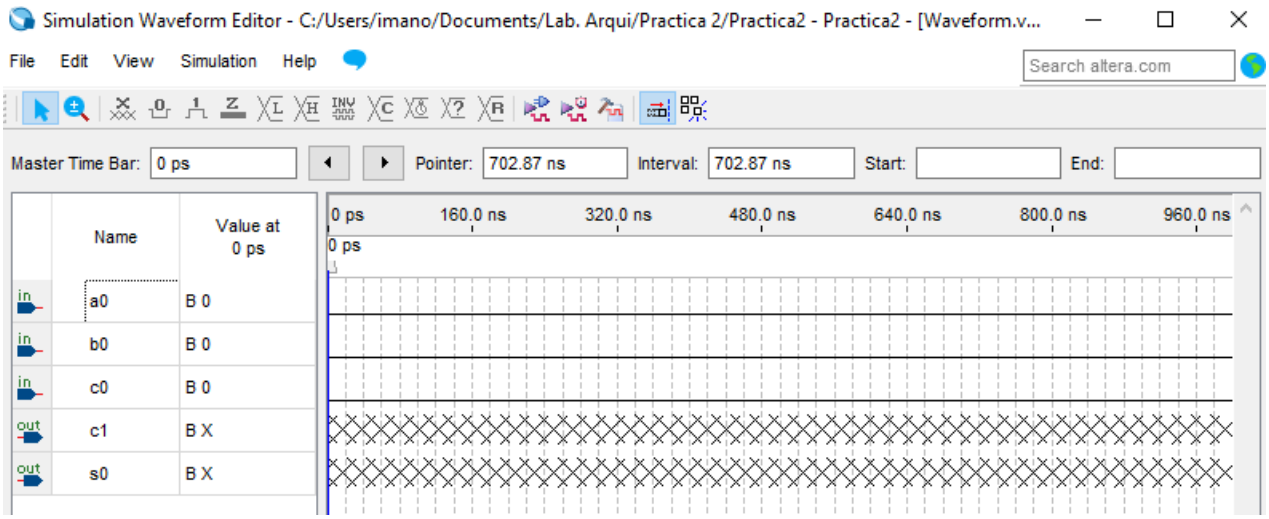
Paso 2: Ármanos el circuito



Simulación 1

Como quiero hacer la simulación en Waveform Editor, entonces seguimos el siguiente paso.

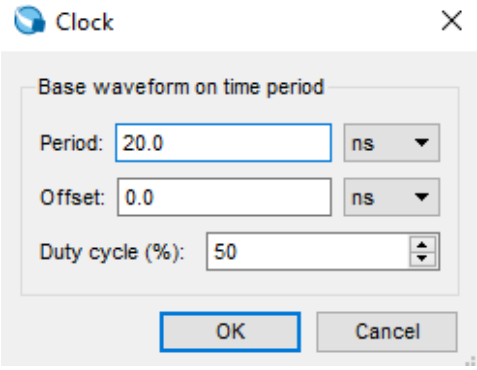
Paso 3: En la ventana New, en la sección Verification/Debugging Files seleccionar la opción University Program VWF.



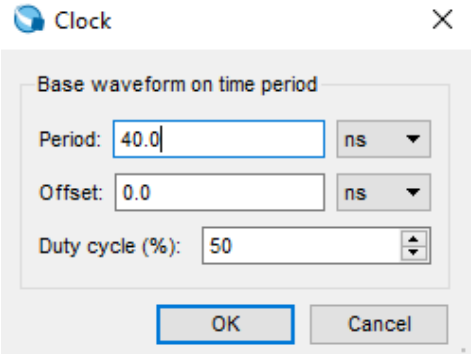
Paso 4: Configurar las entradas.

Ejemplo para probar todas las combinaciones:

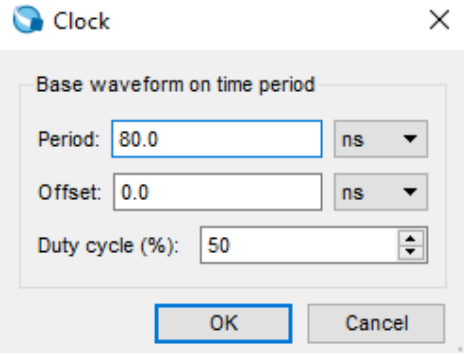
- `a0`: configúralo como **Clock** con periodo 20.



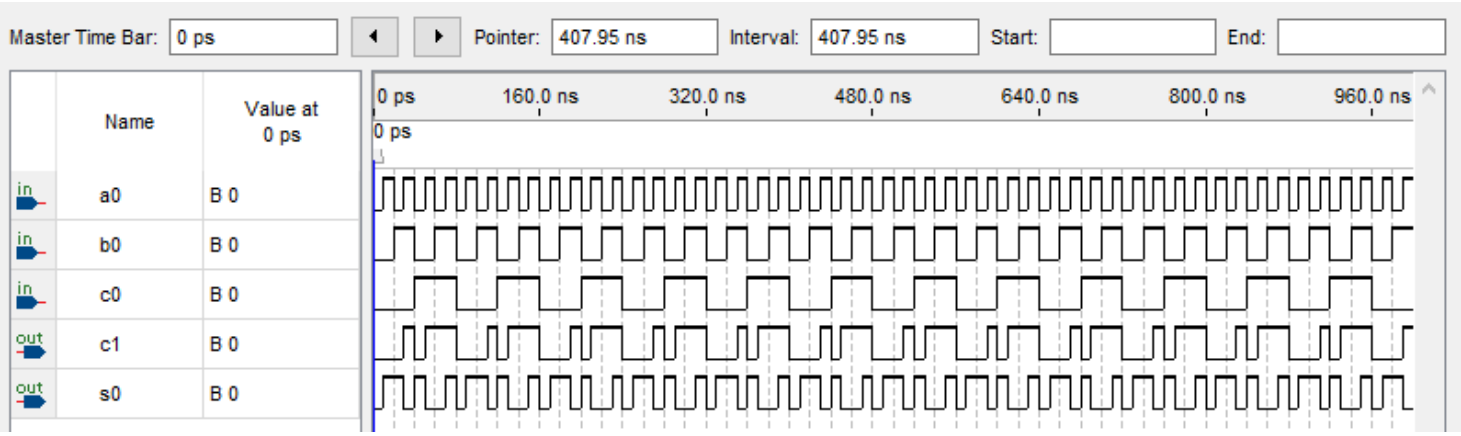
- `b0`: configúralo como **Clock** con periodo 40.



- `c0`: configúralo como **Clock** con periodo 80.



Esto hará que se recorran automáticamente las 8 combinaciones posibles de (a0, b0, c0).



Resultado: en la simulación verás cómo `s0` y `c1` van cambiando según la tabla de verdad del sumador completo.

1. Qué significa configurar como Clock

Cuando configuras una señal como **Clock**, Quartus/ModelSim hace que esa entrada sea una onda periódica que alterna entre `0` y `1` según el **periodo** que le pongas.

- Periodo 20 = cambia más rápido (alta frecuencia).
- Periodo 80 = cambia más lento (baja frecuencia).

2. Cómo funciona en tu ejemplo

- `a0` con periodo **20** → cambia de 0 a 1 cada 10 unidades de tiempo.
- `b0` con periodo **40** → cambia cada 20 unidades de tiempo (la mitad de rápido que `a0`).
- `c0` con periodo **80** → cambia cada 40 unidades de tiempo (todavía más lento).

Esto genera una especie de **contador binario automático**:

Tiempo	c0	b0	a0
0	0	0	0
10	0	0	1
20	0	1	0
30	0	1	1
40	1	0	0
50	1	0	1
60	1	1	0
70	1	1	1

Se generan automáticamente las **8 combinaciones posibles** de las tres entradas `(a0, b0, c0)`.

3. Qué pasa en las salidas

El sumador completo hace:

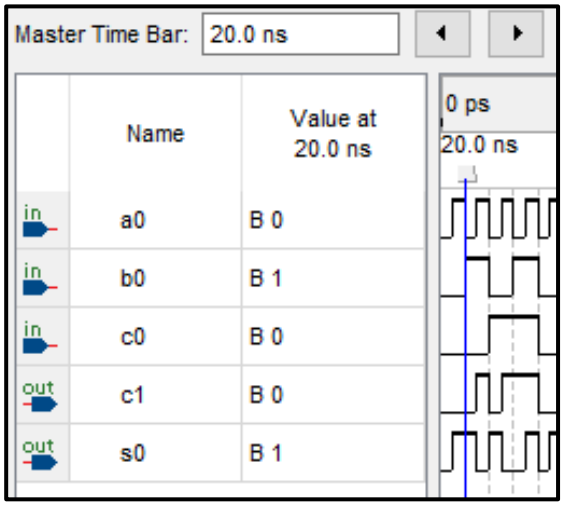
1. `s0 <= a0 xor b0 xor c0` → la suma binaria (bit de suma).
2. `c1 <= (a0 and b0) or (a0 and c0) or (b0 and c0)` → el acarreo.

Al correr la simulación, veremos que las salidas (`s0` y `c1`) se van actualizando exactamente igual que en la **tabla de verdad**.

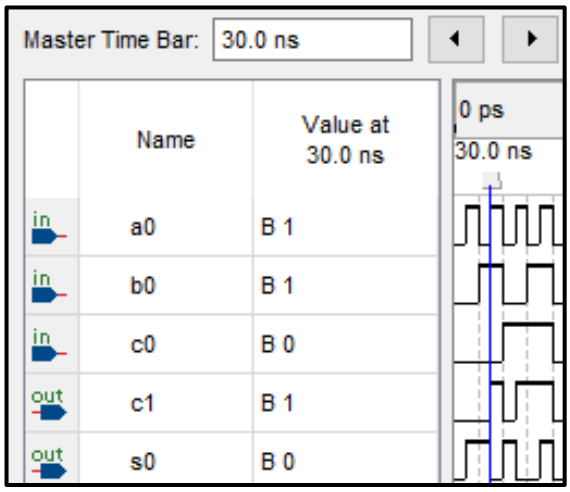
Configurar `a0`, `b0` y `c0` como **Clocks con distintos periodos (20, 40 y 80)** permite que el simulador recorra todas las combinaciones posibles **automáticamente**, y así puedes verificar de una sola corrida que tu diseño del sumador está correcto.

Tiempo	c0	b0	a0	s0 (Suma)	c1 (Acarreo)
0	0	0	0	0	0
10	0	0	1	1	0
20	0	1	0	1	0
30	0	1	1	0	1
40	1	0	0	1	0
50	1	0	1	0	1
60	1	1	0	0	1
70	1	1	1	1	1

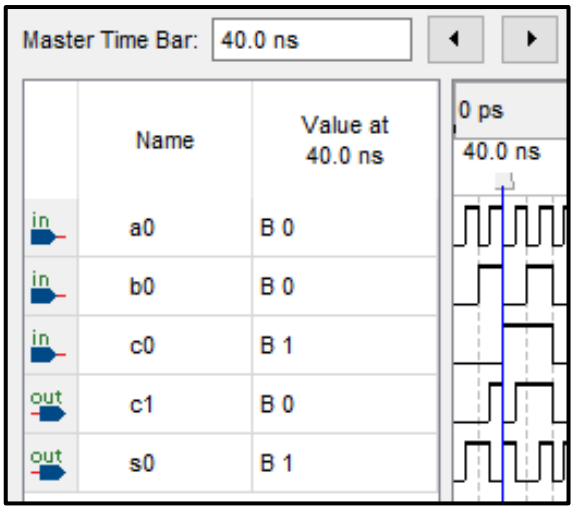
Para confirmar los resultados teoricos, hay que verlo en el simulador, estos son algunos ejemplos:



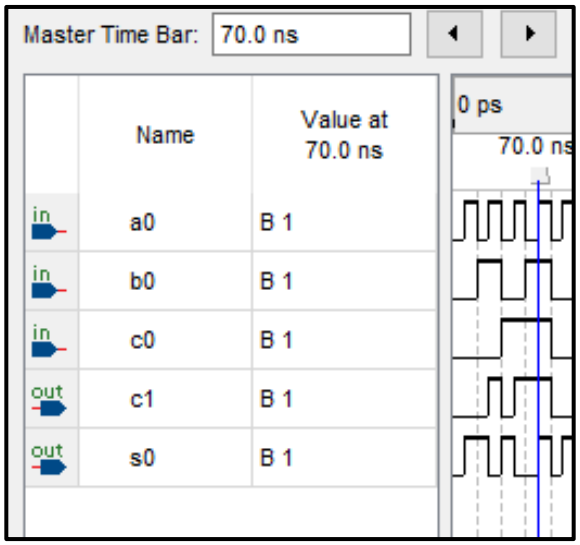
Tiempo	c0	b0	a0	s0 (Suma)	c1 (Acarreo)
20	0	1	0	1	0



Tiempo	c0	b0	a0	s0 (Suma)	c1 (Acarreo)
30	0	1	1	0	1



Tiempo	c0	b0	a0	s0 (Suma)	c1 (Acarreo)
40	1	0	0	1	0



Tiempo	c0	b0	a0	s0 (Suma)	c1 (Acarreo)
70	1	1	1	1	1

4. Cómo interpretarlo

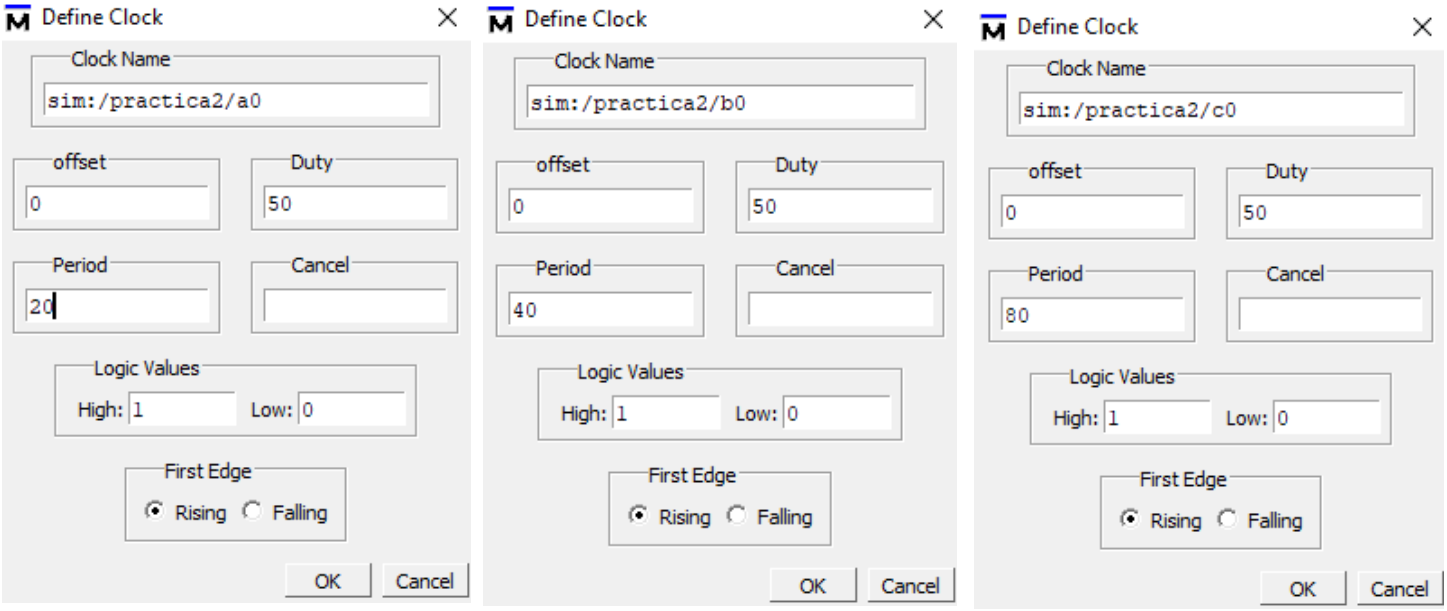
- `s0` es la **suma de un solo bit** → se calcula con XOR: **s0 = a0 ⊕ b0 ⊕ c0**
- `c1` es el **acarreo de salida** → se calcula con la fórmula: **c1 = (a0·b0) + (a0·c0) + (b0·c0)**

En conclusión, en la simulación con los **Clocks configurados (20, 40 y 80)** deberiamos ver que los valores de `s0` y `c1` cambian siguiendo exactamente esta tabla, y en una sola corrida recorres todas las combinaciones.

Simulación 2

Utilizamos la otra simulación que es la misma interpretación

En esta imagen vemos las entradas y salidas de la simulación. Ahora toco asignarles los mismo valores de entrada para a0, b0, c0, como se muestra a continuación:



Se configuran con los mismo periodos de 20, 40 y 80.

Como vemos estos resultados del segundo simulador corresponden a los mostrado en la tabla de verdad para cada intervalo de tiempo que corresponda.

Conclusión

La implementación del sumador completo de 1 bit permitió validar el flujo de diseño en Quartus, desde la descripción en VHDL hasta la verificación funcional. A través del **RTL Viewer**, se corroboró la síntesis lógica del circuito, mientras que el uso de señales de reloj en el **Simulation Waveform Editor** optimizó la verificación de la tabla de verdad al automatizar la transición de estados. A pesar de los retos iniciales en la sincronización de periodos de simulación, la concordancia entre los resultados obtenidos y el comportamiento teórico del sumador confirma una sólida comprensión de las herramientas de simulación, estableciendo un precedente metodológico para el diseño de sistemas digitales de mayor escala.

Referencias

1. Intel. (2025). *Intel® Quartus® Prime Design Software*. Intel. Recuperado el 10 de septiembre de 2025 de https://www.intel.com/content/www/us/en/software/programmable/quartus-prime/overview.html
2. FPGA4student. (2025). *VHDL tutorial: Full Adder*. FPGA4student. Recuperado el 10 de septiembre de 2025 de https://www.fpga4student.com/2017/01/vhdl-code-for-full-adder.html
3. Allaboutcircuits. (2025). *Introduction to VHDL for Digital Design*. All About Circuits. Recuperado el 10 de septiembre de 2025 de https://www.allaboutcircuits.com/technical-articles/introduction-to-vhdl-for-digital-design
4. DigiKey. (2025). *Getting Started with Quartus Prime and VHDL*. DigiKey. Recuperado el 10 de septiembre de 2025 de https://www.digikey.com/en/maker/projects/getting-started-with-quartus-prime-and-vhdl